VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

THAMMISETTY JAHNAVI(1BF24CS317)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by Thammisetty Jahnavi(1BF24CS317), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026

. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

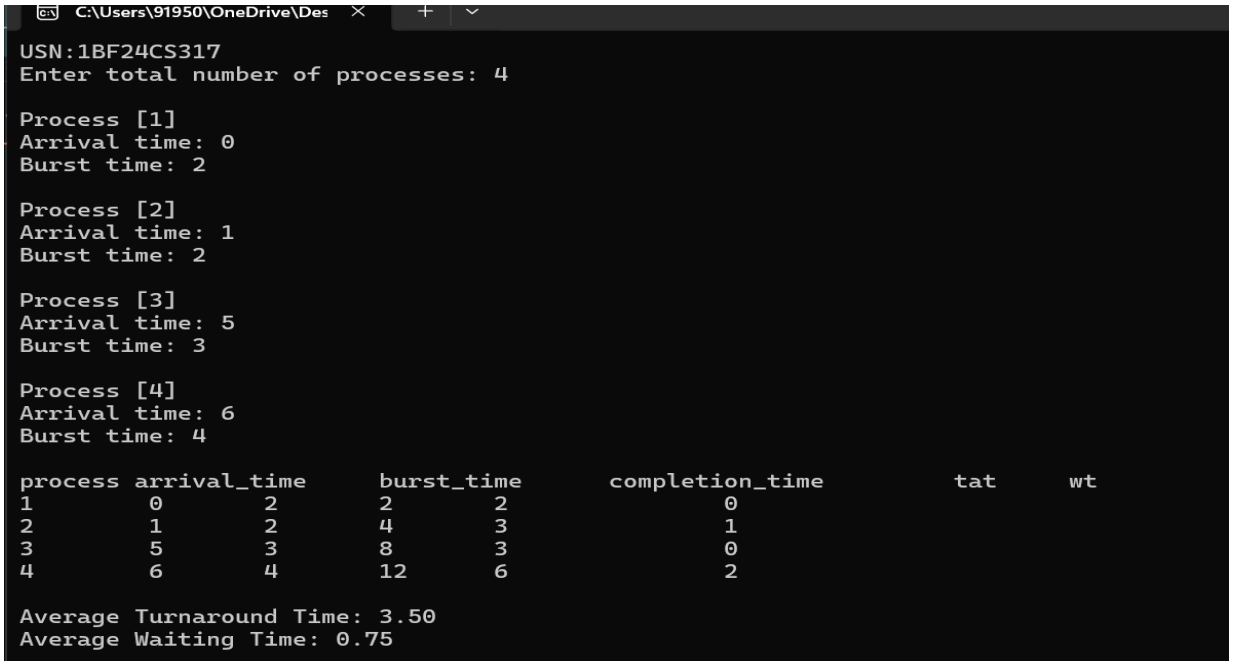
Code:

=>FCFS:

```
#include<stdio.h>
int main(){
int n,i,j;

   printf("USN:1BF24CS317\n");
   printf("Enter total number of processes: ");
   scanf("%d", &n);

   int pid[n], at[n], bt[n], ct[n], tat[n], wt[n];


   for(i = 0; i < n; i++) {
      pid[i] = i + 1;
      printf("\nProcess [%d]\n", pid[i]);
      printf("Arrival time: ");
      scanf("%d", &at[i]);
      printf("Burst time: ");
      scanf("%d", &bt[i]);
   }
   int curr_time=0;
   for(i=0;i<n-1;i++){
      for(j=i+1;j<n;j++){
         if(at[i]>at[j]){
               int temp;
            temp = at[i];
            at[i] = at[j];
            at[j] = temp;
```

```
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
            temp = pid[i];
            pid[i] = pid[j];
            pid[j] = temp;

        }
    }
}
for(int i=0;i<n;i++){
    if(curr_time<at[i]){
      curr_time=at[i];
    }
    ct[i]=curr_time+bt[i];
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];
    curr_time=ct[i];
}
float atat=0,awt=0;
for(i=0;i<n;i++){
    atat+=tat[i];
    awt+=wt[i];
}
atat/=n;
awt/=n;
printf("\nprocess\tarrival_time\t burst_time\t completion_time\t tat\t wt\n");
for(i=0;i<n;i++){
    printf("%d\t %d\t %d\t %d\t %d\t\t %d\t\t\n",pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
}

printf("\nAverage Turnaround Time: %.2f", atat);
printf("\nAverage Waiting Time: %.2f\n", awt);
}
```

Result:



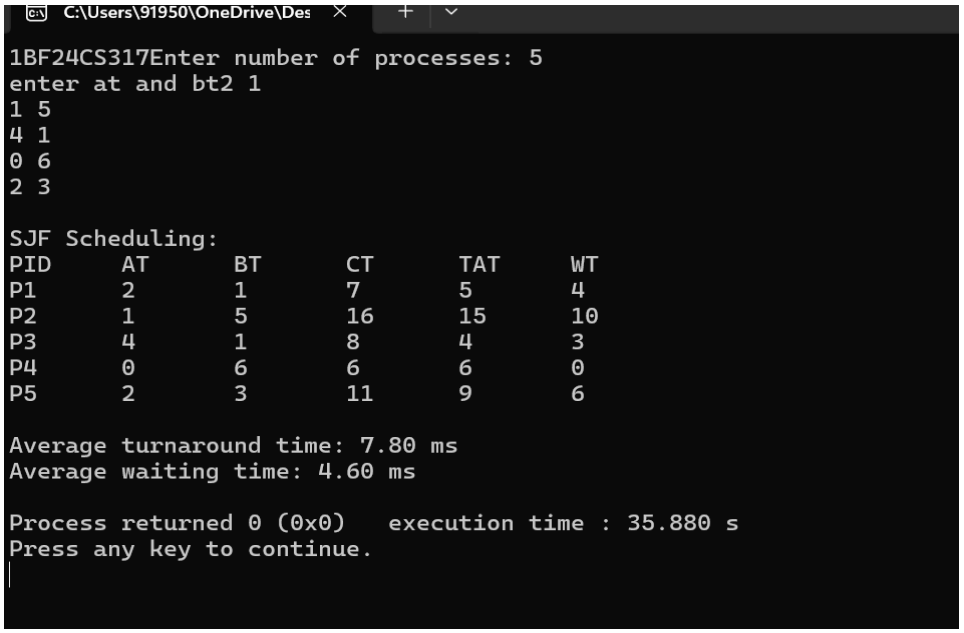
=>SJF(Non-preemptive):

```
#include<stdio.h>
int main(){
int n;
   printf("1BF24CS317");

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int pid[n], at[n], bt[n], ct[n], tat[n], wt[n];
   int finished[n];
   for(int i=0;i<n;i++){
      pid[i]=i+1;
      finished[i]=0;
}
```

```
printf("enter at and bt");
for(int i=0;i<n;i++){
   scanf("%d",&at[i]);
   scanf("%d",&bt[i]);
}
int current_time = 0;
int completed = 0;

while(completed < n){
   int idx = -1;
   int min_bt = 9999;

   for(int i=0;i<n;i++){
      if(at[i] <= current_time && finished[i] == 0){
         if(bt[i] < min_bt){
            min_bt = bt[i];
            idx = i;
         }
      }
   }

   if(idx == -1){
      current_time++;
   }
   else{
      ct[idx] = current_time + bt[idx];
      tat[idx] = ct[idx] - at[idx];
      wt[idx] = tat[idx] - bt[idx];

      current_time = ct[idx];
      finished[idx] = 1;
```

```
            completed++;
        }
    }

    float avg_tat = 0, avg_wt = 0;

    printf("\nSJF Scheduling:\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

    for(int i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);

        avg_tat += tat[i];
        avg_wt += wt[i];
    }
    avg_tat /= n;
    avg_wt /= n;

    printf("\nAverage turnaround time: %.2f ms\n",avg_tat);
    printf("Average waiting time: %.2f ms\n",avg_wt);
    return 0;
}
```

Result:



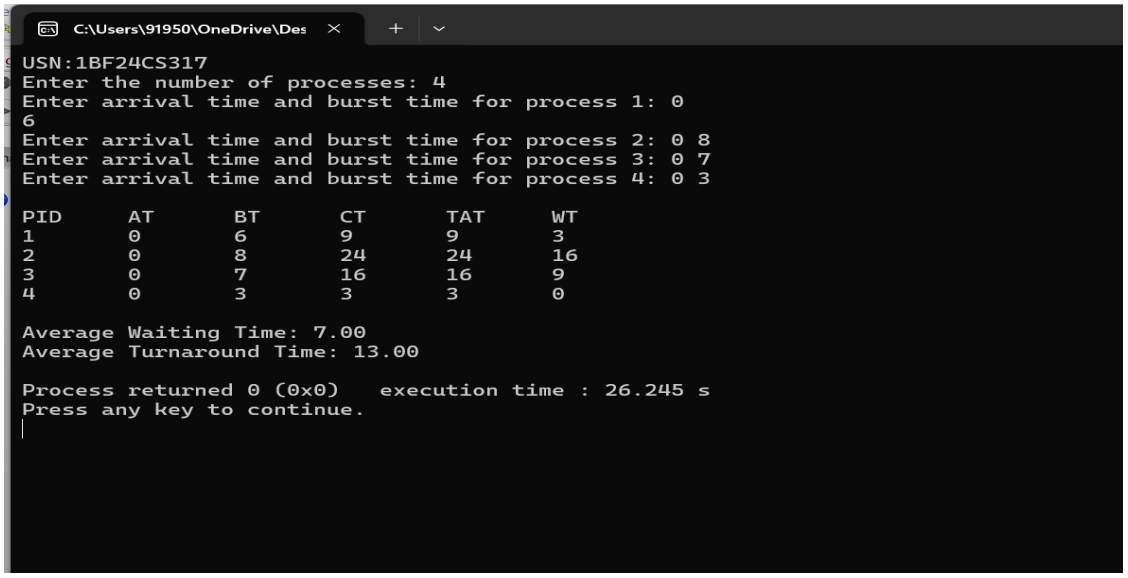
=>SJF(Preemptive):

```
#include <stdio.h>
#include <limits.h>

int main() {
    int n;

    printf("Enter the number of processes: ");
    scanf("%d",&n);

    int pid[n], at[n], bt[n], rt[n];
    int ct[n], tat[n], wt[n];

    for(int i=0;i<n;i++){
        pid[i] = i+1;
    }
```

```
for(int i=0;i<n;i++){
    printf("Enter arrival time and burst time for process %d: ",i+1);
    scanf("%d %d",&at[i],&bt[i]);
    rt[i] = bt[i];
}

int completed = 0;
int t = 0;
while(completed < n){
    int idx = -1;
    int min_rt = INT_MAX;
    for(int i=0;i<n;i++){
        if(at[i] <= t && rt[i] > 0){
            if(rt[i] < min_rt ||
               (rt[i] == min_rt && at[i] < at[idx])){
                min_rt = rt[i];
                idx = i;
            }
        }
    }
    if(idx == -1){
        t++;
        continue;
    }
    rt[idx]--;
    t++;
    if(rt[idx] == 0){
        completed++;
        ct[idx] = t;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];
```

```
        if(wt[idx] < 0)
            wt[idx] = 0;
    }
  }

  float avg_wt = 0, avg_tat = 0;
  printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
  for(int i=0;i<n;i++){
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
          pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    avg_wt += wt[i];
    avg_tat += tat[i];
  }

  printf("\nAverage Waiting Time: %.2f",avg_wt/n);
  printf("\nAverage Turnaround Time: %.2f\n",avg_tat/n);

  return 0;
}
```

Result:



Program -2

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
→ Priority (pre-emptive & Non-pre-emptive)
→Round Robin (Experiment with different quantum sizes for RR algorithm)

Code:

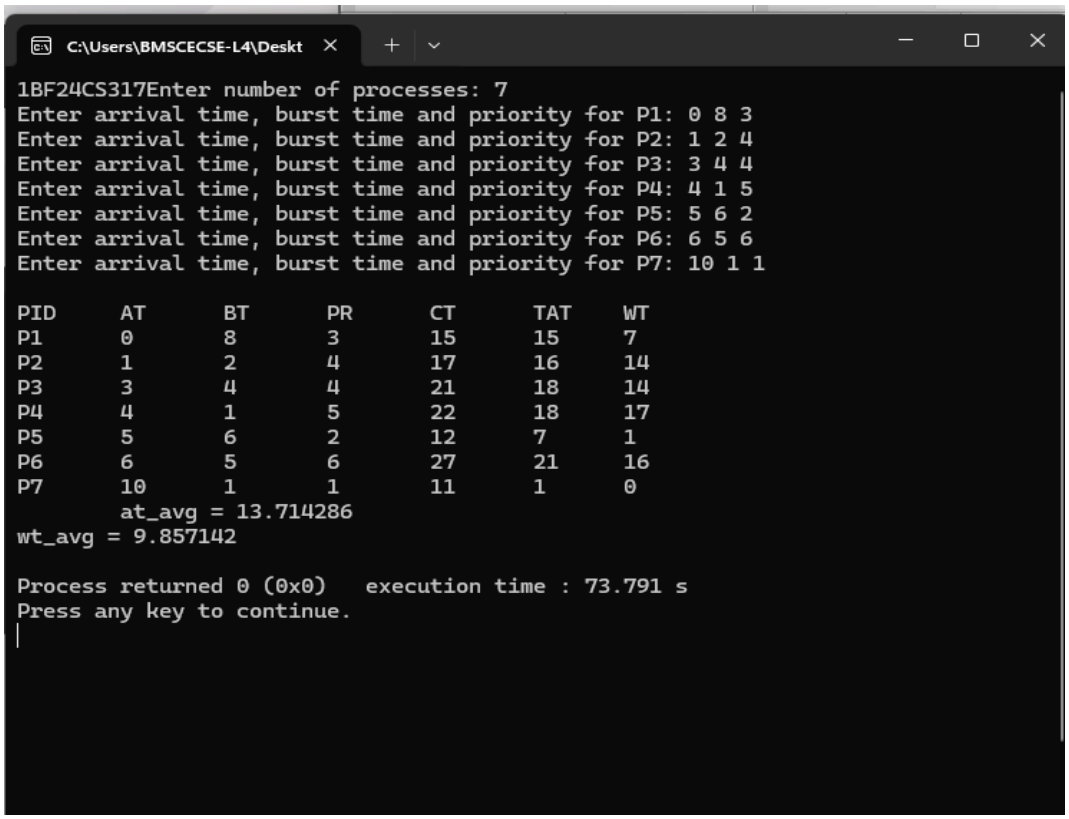
=>Priority (pre-emptive):

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n;
    printf("1BF24CS317");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], pr[n];
    int rt[n], ct[n], tat[n], wt[n];

    for(int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter arrival time, burst time and priority for P%d: ", pid[i]);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rt[i] = bt[i];
    }

    int current_time = 0;
    int completed = 0;

    while(completed < n) {
        int highest_priority = 9999;
        int selected = -1;

        for(int i = 0; i < n; i++) {
            if(at[i] <= current_time && rt[i] > 0) {
```

```
            if(pr[i] < highest_priority) {
                highest_priority = pr[i];
                selected = i;
            }
        }
    }

    if(selected == -1) {
        current_time++;
    }
    else {
        rt[selected]--;
        current_time++;

        if(rt[selected] == 0) {
            ct[selected] = current_time;

            tat[selected] = ct[selected] - at[selected];

            wt[selected] = tat[selected] - bt[selected];

            completed++;
        }
    }
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

float avg_tat = 0, avg_wt = 0;

for(int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);

    avg_tat += tat[i];
    avg_wt += wt[i];
}

printf("\tat_avg = %f", avg_tat / n);
printf("\nwt_avg = %f\n", avg_wt / n);

return 0;
```

}

Result:



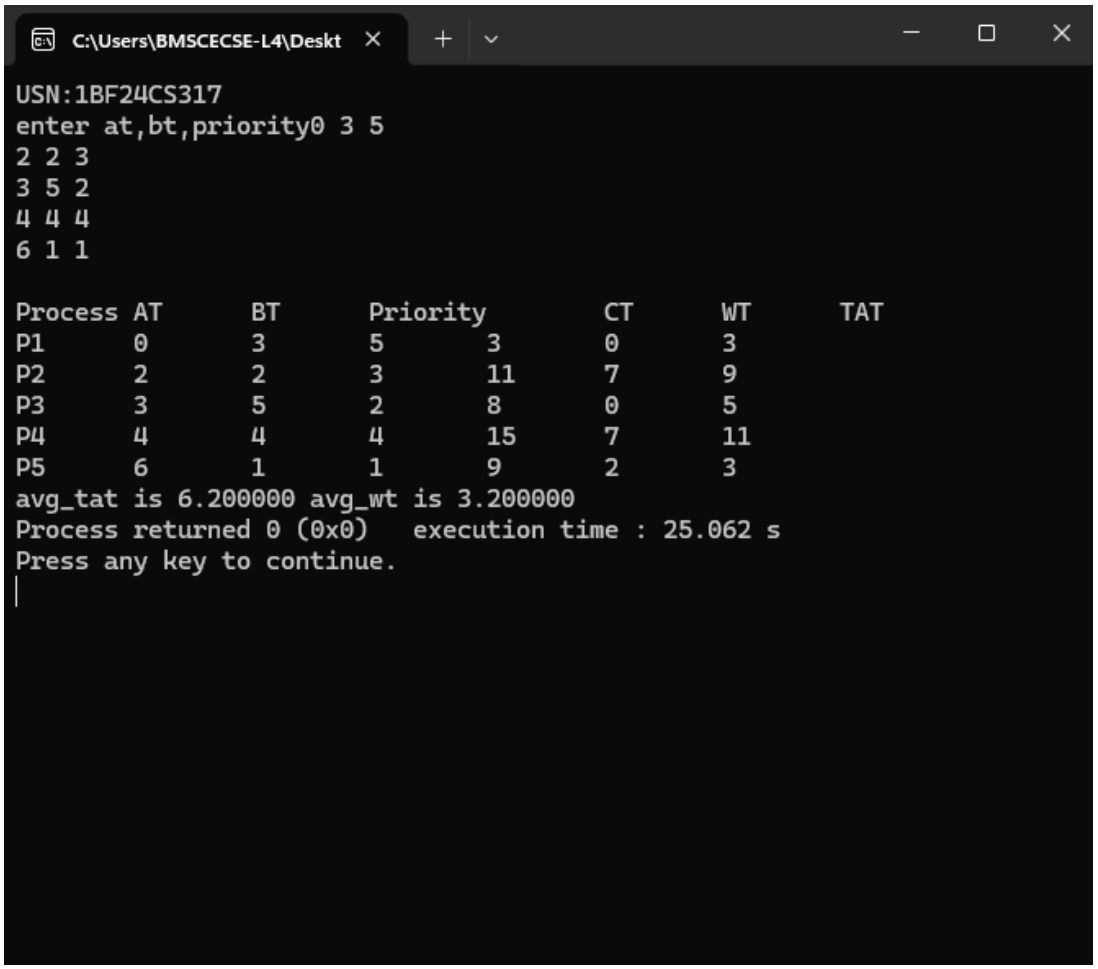
=>Priority (Non-preemptive):

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    printf("USN:1BF24CS317");
   int n = 5;

   float tat=0;
   float wt=0;
   int process_id[5] = {1,2,3,4,5};
   int arrival_time[5];
   int burst_time[5];
   int priority[5] ;
   printf("enter at,bt,priority");
```

```
for(int i=0;i<n;i++){
   scanf("%d",&arrival_time[i]);
   scanf("%d",&burst_time[i]);
   scanf("%d",&priority[i]);
}
for(int i=0;i<n;i++){

}

int completion_time[5], turnaround_time[5], waiting_time[5];
bool completed[5];

int current_time = 0;
int completed_count = 0;

for(int i = 0; i < n; i++)
   completed[i] = false;

while(completed_count < n) {

   int highest_priority = 9999;
   int selected_process = -1;

   for(int i = 0; i < n; i++) {
      if(arrival_time[i] <= current_time && completed[i] == false) {
         if(priority[i] < highest_priority) {
            highest_priority = priority[i];
            selected_process = i;
         }
      }
   }

   if(selected_process == -1) {
      current_time++;
   }
   else {
      int start_time = current_time;

      completion_time[selected_process] =
         start_time + burst_time[selected_process];

      turnaround_time[selected_process] =
```

```
            completion_time[selected_process] -
            arrival_time[selected_process];

        waiting_time[selected_process] =
            turnaround_time[selected_process] -
            burst_time[selected_process];

        current_time = completion_time[selected_process];

        completed[selected_process] = true;
        completed_count++;
      }
    }

    printf("\nProcess\tAT\tBT\tPriority\tCT\tWT\tTAT\n");

    for(int i = 0; i < n; i++) {
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
      process_id[i],
      arrival_time[i],
      burst_time[i],
      priority[i],
      completion_time[i],
      waiting_time[i],
      turnaround_time[i]);
    }
    for(int i=0;i<n;i++){
      tat+=turnaround_time[i];
      wt+=waiting_time[i];
    }
    tat/=n;
    wt/=n;
    printf("avg_tat is %f avg_wt is %f ",tat,wt);

    return 0;
}
```

Result:



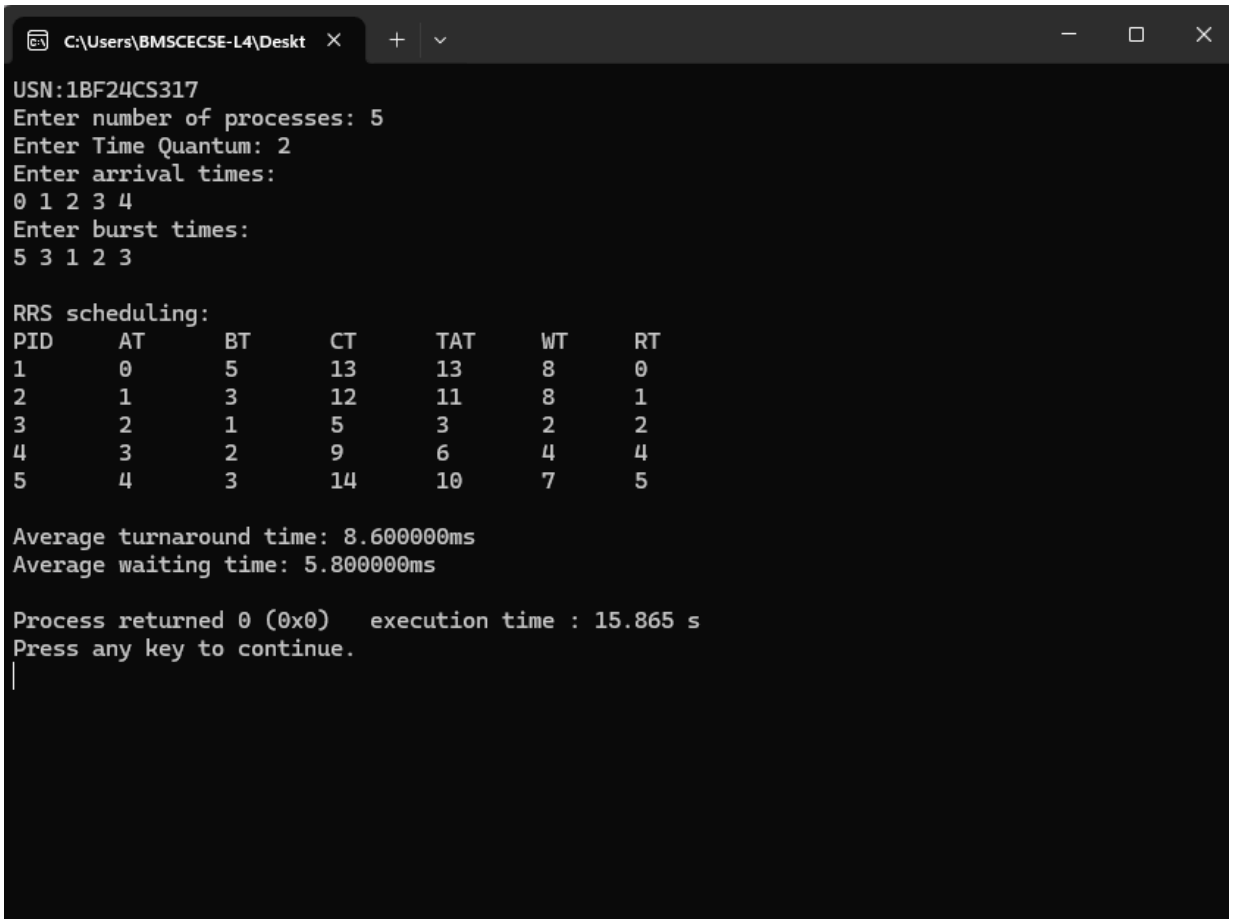
=>Round Robin:

```
#include <stdio.h>

int main() {
    int n, tq;
    printf("USN:1BF24CS317");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], rt_resp[n];
    int queue[100], front = 0, rear = 0;
```

```
int visited[n];

printf("Enter arrival times:\n");
for(int i = 0; i < n; i++)
   scanf("%d", &at[i]);

printf("Enter burst times:\n");
for(int i = 0; i < n; i++) {
   scanf("%d", &bt[i]);
   rt[i] = bt[i];
   visited[i] = 0;
   rt_resp[i] = -1;
}

int current_time = 0, completed = 0;

queue[rear++] = 0;
visited[0] = 1;

while(front < rear) {

   int i = queue[front++];

   if(rt_resp[i] == -1)
      rt_resp[i] = current_time - at[i];

   if(rt[i] > tq) {
      current_time += tq;
      rt[i] -= tq;
   }
   else {
      current_time += rt[i];
      rt[i] = 0;

      ct[i] = current_time;
      tat[i] = ct[i] - at[i];
      wt[i] = tat[i] - bt[i];

      completed++;
   }

   for(int j = 0; j < n; j++) {
```

```
            if(at[j] <= current_time && visited[j] == 0) {
                queue[rear++] = j;
                visited[j] = 1;
            }
        }

        if(rt[i] > 0)
            queue[rear++] = i;

        if(front == rear) {
            for(int j = 0; j < n; j++) {
                if(rt[j] > 0) {
                    queue[rear++] = j;
                    visited[j] = 1;
                    break;
                }
            }
        }
    }

    float avg_wt = 0, avg_tat = 0;

    printf("\nRRS scheduling:\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for(int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
              i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt_resp[i]);

        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage turnaround time: %.6fms\n", avg_tat/n);
    printf("Average waiting time: %.6fms\n", avg_wt/n);

    return 0;
}
```

Result:



Program -3:

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

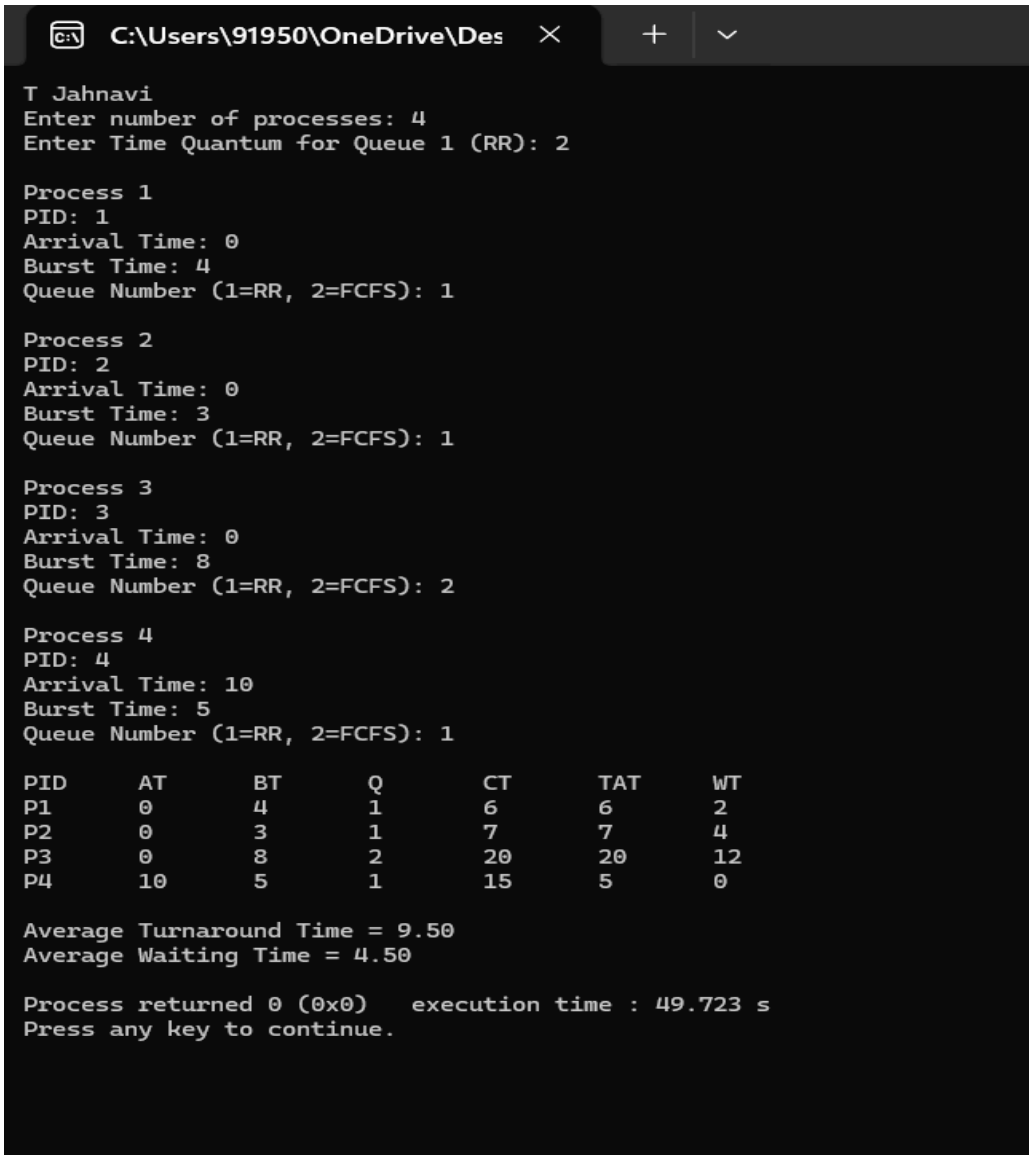
```
#include <stdio.h>

#define MAX 50
```

```
typedef struct {
   int pid, at, bt, rt, ct, tat, wt, qno;
} Process;

int main() {
   printf("T Jahnavi\n");
   Process p[MAX];
   int n, i, time = 0, completed = 0;
   int tq;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter Time Quantum for Queue 1 (RR): ");
   scanf("%d", &tq);

   for (i = 0; i < n; i++) {
      printf("\nProcess %d\n", i + 1);

      printf("PID: ");
      scanf("%d", &p[i].pid);

      printf("Arrival Time: ");
      scanf("%d", &p[i].at);

      printf("Burst Time: ");
      scanf("%d", &p[i].bt);

      printf("Queue Number (1=RR, 2=FCFS): ");
      scanf("%d", &p[i].qno);

      p[i].rt = p[i].bt;
   }

   while (completed < n) {

      int executed = 0;

      for (i = 0; i < n; i++) {
         if (p[i].qno == 1 && p[i].at <= time && p[i].rt > 0) {

            executed = 1;
```

```
            int run = (p[i].rt > tq) ? tq : p[i].rt;

            time += run;
            p[i].rt -= run;

            if (p[i].rt == 0) {
                p[i].ct = time;
                completed++;
            }
        }
    }


    if (!executed) {
        for (i = 0; i < n; i++) {
            if (p[i].qno == 2 && p[i].at <= time && p[i].rt > 0) {

                executed = 1;


                time++;
                p[i].rt--;

                if (p[i].rt == 0) {
                    p[i].ct = time;
                    completed++;
                }

                break;
            }
        }
    }

    // CPU Idle
    if (!executed) {
        time++;
    }
}


float total_tat = 0, total_wt = 0;
```

```
    printf("\nPID\tAT\tBT\tQ\tCT\tTAT\tWT\n");

    for (i = 0; i < n; i++) {
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;

        total_tat += p[i].tat;
        total_wt += p[i].wt;

        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].at, p[i].bt, p[i].qno,
               p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

    return 0;
}
```

Result:



Program -4:
Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms:
a) Rate- Monotonic
b) Earliest-deadline First
c) Proportional scheduling

Code:

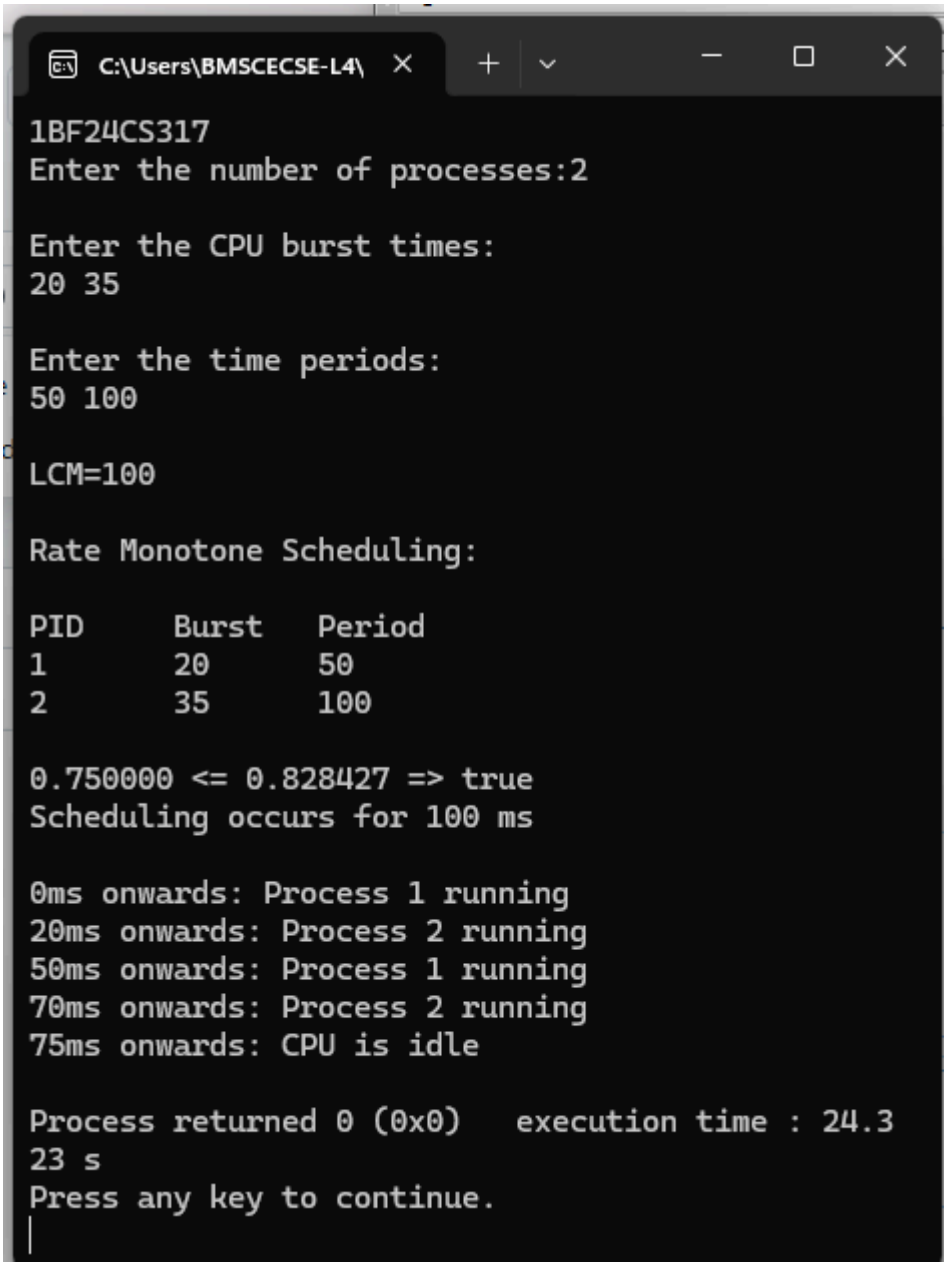
=> Rate- Monotonic

```
#include <stdio.h>
#include <math.h>

#define MAX 10

int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int find_lcm(int arr[], int n) {
    int res = arr[0];
    for (int i = 1; i < n; i++) {
        res = lcm(res, arr[i]);
    }
    return res;
}

int main() {
    printf("1BF24CS317\nR");
    int n, C[MAX], P[MAX], remaining[MAX] = {0};
    int hyperperiod;
    float utilization = 0.0;

    printf("Enter the number of processes:");
    scanf("%d", &n);

    printf("\nEnter the CPU burst times:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &C[i]);
```

```
}

printf("\nEnter the time periods:\n");
for (int i = 0; i < n; i++) {
   scanf("%d", &P[i]);
}

hyperperiod = find_lcm(P, n);
printf("\nLCM=%d\n\n", hyperperiod);

printf("Rate Monotone Scheduling:\n\n");
printf("PID\tBurst\tPeriod\n");
for (int i = 0; i < n; i++) {
   printf("%d\t%d\t%d\n", i + 1, C[i], P[i]);
}


for (int i = 0; i < n; i++) {
   utilization += (float)C[i] / P[i];
}

float bound = n * (pow(2, 1.0/n) - 1);

printf("\n%f <= %f => %s\n",
      utilization, bound,
      (utilization <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n\n", hyperperiod);

int prev_task = -1;

for (int t = 0; t < hyperperiod; t++) {


   for (int i = 0; i < n; i++) {
      if (t % P[i] == 0) {
         remaining[i] = C[i];
      }
   }
   int task = -1;
   int minP = 9999;
```

```
        for (int i = 0; i < n; i++) {
            if (remaining[i] > 0 && P[i] < minP) {
                minP = P[i];
                task = i;
            }
        }

        if (task != prev_task) {
            if (task == -1)
                printf("%dms onwards: CPU is idle\n", t);
            else
                printf("%dms onwards: Process %d running\n", t, task + 1);
        }
        if (task != -1) {
            remaining[task]--;
        }

        prev_task = task;
    }

    return 0;
}
```

Result:



=> Earliest-deadline First

Code:

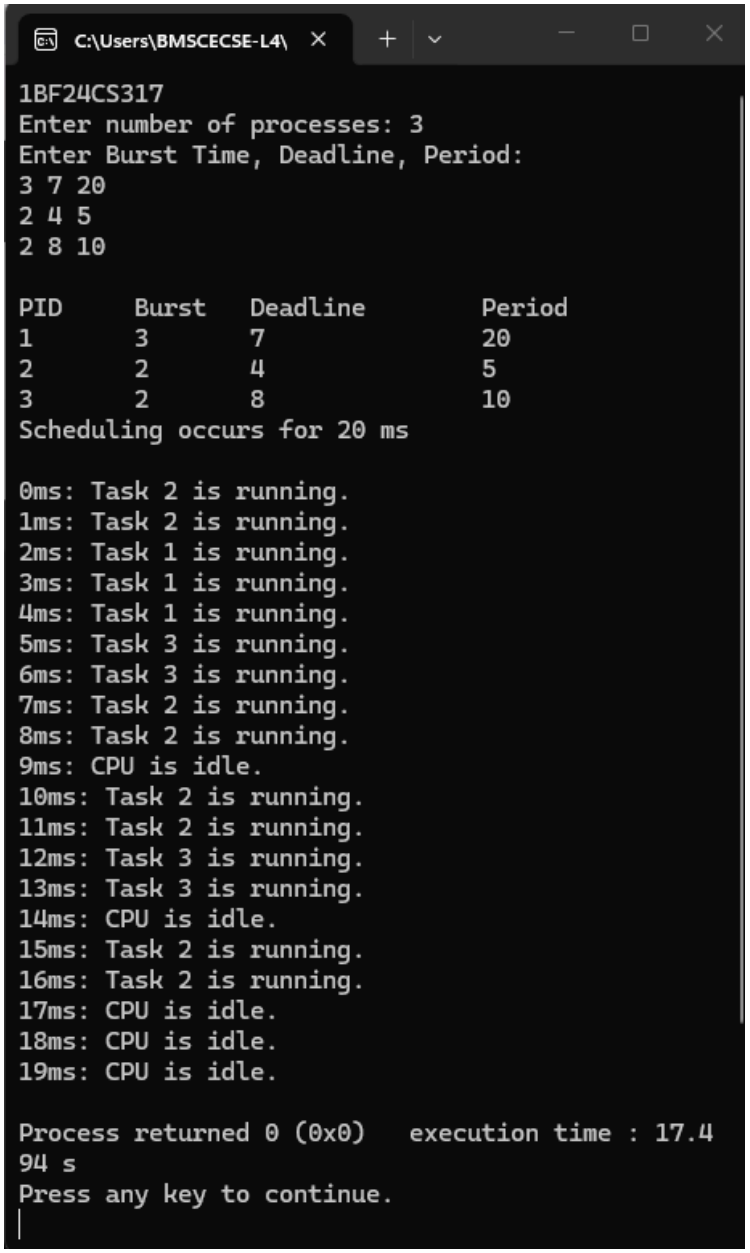
```
#include <stdio.h>

#define MAX 10
#define INF 999999
```

```
int gcd(int a, int b) {
   if (b == 0) return a;
   return gcd(b, a % b);
}


int lcm(int a, int b) {
   return (a * b) / gcd(a, b);
}


int find_lcm(int arr[], int n) {
   int res = arr[0];
   for (int i = 1; i < n; i++) {
      res = lcm(res, arr[i]);
   }
   return res;
}

int main() {
   int n, i;
   printf("1BF24CS317\n");

   int burst[MAX], deadline[MAX], period[MAX];
   int remaining[MAX];
   int abs_deadline[MAX];

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter Burst Time, Deadline, Period:\n");
   for (i = 0; i < n; i++) {
      scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
      remaining[i] = 0;
      abs_deadline[i] = INF;
   }


   int hyper = find_lcm(period, n);
```

```
printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (i = 0; i < n; i++) {
   printf("%d\t%d\t%d\t\t%d\n", i + 1, burst[i], deadline[i], period[i]);
}

printf("Scheduling occurs for %d ms\n\n", hyper);


for (int time = 0; time < hyper; time++) {


   for (i = 0; i < n; i++) {
      if (time % period[i] == 0) {
         remaining[i] = burst[i];
         abs_deadline[i] = time + deadline[i];
      }
   }

   int selected = -1;
   int min_deadline = INF;


   for (i = 0; i < n; i++) {
      if (remaining[i] > 0) {
         if (abs_deadline[i] < min_deadline) {
            min_deadline = abs_deadline[i];
            selected = i;
         }
      }
   }


   if (selected == -1) {
      printf("%dms: CPU is idle.\n", time);
   } else {
      printf("%dms: Task %d is running.\n", time, selected + 1);

      remaining[selected]--;

      if (remaining[selected] == 0) {
         abs_deadline[selected] = INF;
      }
```

```
        }
    }

    return 0;
}
```

Result:

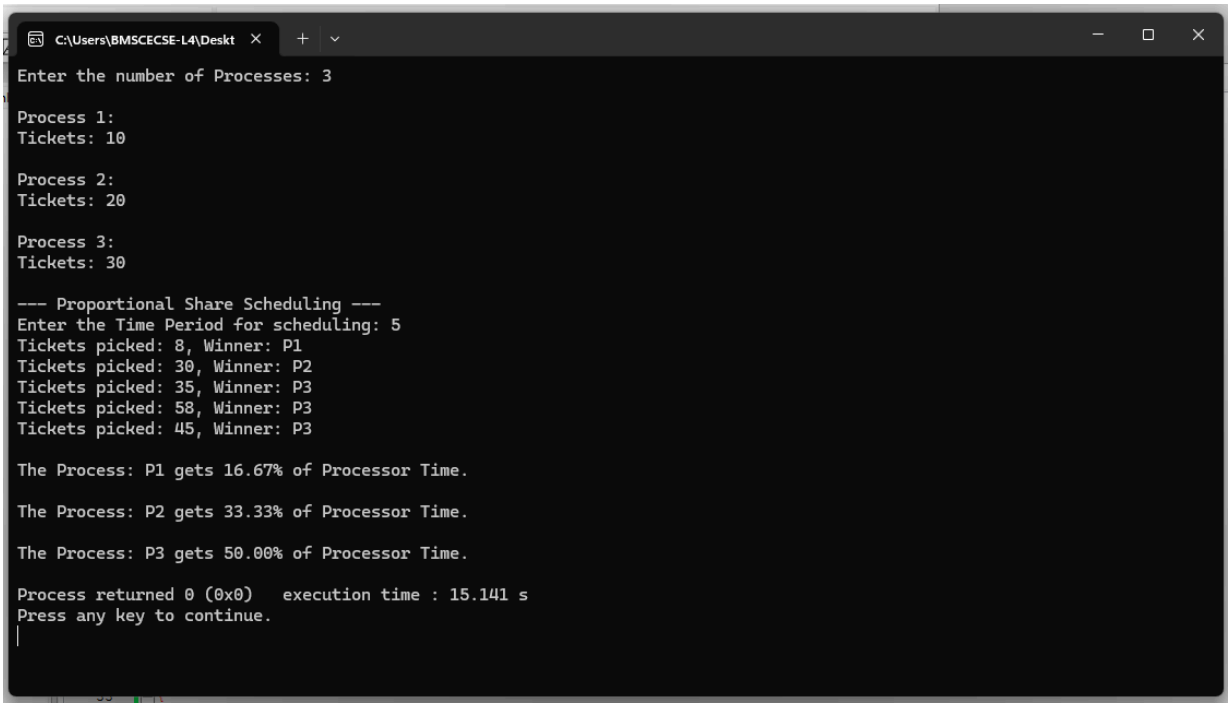


=>Proportional scheduling
Code:

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct
{
char name[5]; int tickets; }
Process;
int main() {
int n, total_tickets = 0; float total_T =0.0;
printf("Enter the number of Processes: ");
scanf("%d", &n);
Process p[n];
srand(time(NULL));
for (int i = 0; i < n; i++)
{ printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T +=p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;

scanf("%d",&m);
for (int i = 0; i < m; i++)
{
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets)
{
winner_index = j; break;
}
}
```

```
printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);
}
for (int i = 0; i < n; i++)
{
printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
((p[i].tickets / total_T) * 100));

}
return 0;
}
```

Result:



Program -5
Question:
Write a C program to simulate producer-consumer problem
using semaphores

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define SIZE 5

int stack[SIZE];
int top = -1;

// Semaphore variables
int empty = SIZE;
int full = 0;
int mutex = 1;

// WAIT operation
void wait(int *s) {
   (*s)--;
}

void signal(int *s) {
   (*s)++;
}


void producer() {
   int item;

   if (empty == 0) {
      printf("\nBuffer is FULL! Producer cannot produce.\n");
      return;
   }

   printf("\n[ENTRY] Enter item to produce: ");
   scanf("%d", &item);

   wait(&empty);
   wait(&mutex);

    top++;
   stack[top] = item;
   printf("[CRITICAL] Produced item %d at position %d\n", item, top);

   signal(&mutex);
   signal(&full);

   printf("[EXIT] Producer done\n");
```

```
}

void consumer() {
    int item;

    if (full == 0) {
        printf("\nBuffer is EMPTY! Consumer cannot consume.\n");
        return;
    }

    wait(&full);
    wait(&mutex);

    item = stack[top];
    printf("[CRITICAL] Consumed item %d from position %d\n", item, top);
    top--;

    signal(&mutex);
    signal(&empty);


    printf("[EXIT] Consumer done\n");
}


void display() {
    int i;
    if (top == -1) {
        printf("\nStack is empty\n");
        return;
    }

    printf("\nStack contents: ");
    for (i = top; i >= 0; i--) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}

int main() {
    int choice;
```

```
    while (1) {
        printf("\n--- Producer Consumer (Stack) ---\n");
        printf("1. Produce\n2. Consume\n3. Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                producer();
                break;
            case 2:
                consumer();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Result:

```
--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 1

[ENTRY] Enter item to produce: 10
[CRITICAL] Produced item 10 at position 0
[EXIT] Producer done

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 1

[ENTRY] Enter item to produce: 20
[CRITICAL] Produced item 20 at position 1
[EXIT] Producer done

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 3

Stack contents: 20 10

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 2
[CRITICAL] Consumed item 20 from position 1
[EXIT] Consumer done

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 3

Stack contents: 10

--- Producer Consumer (Stack) ---
```

```
--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 3

Stack contents: 10

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 2
[CRITICAL] Consumed item 10 from position 0
[EXIT] Consumer done

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 2

Buffer is EMPTY! Consumer cannot consume.

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 4

Process returned 0 (0x0)   execution time : 1611.640 s
Press any key to continue.
```

Program 6
Question:

Write a C program to simulate the concept of Dining Philosophers problem.

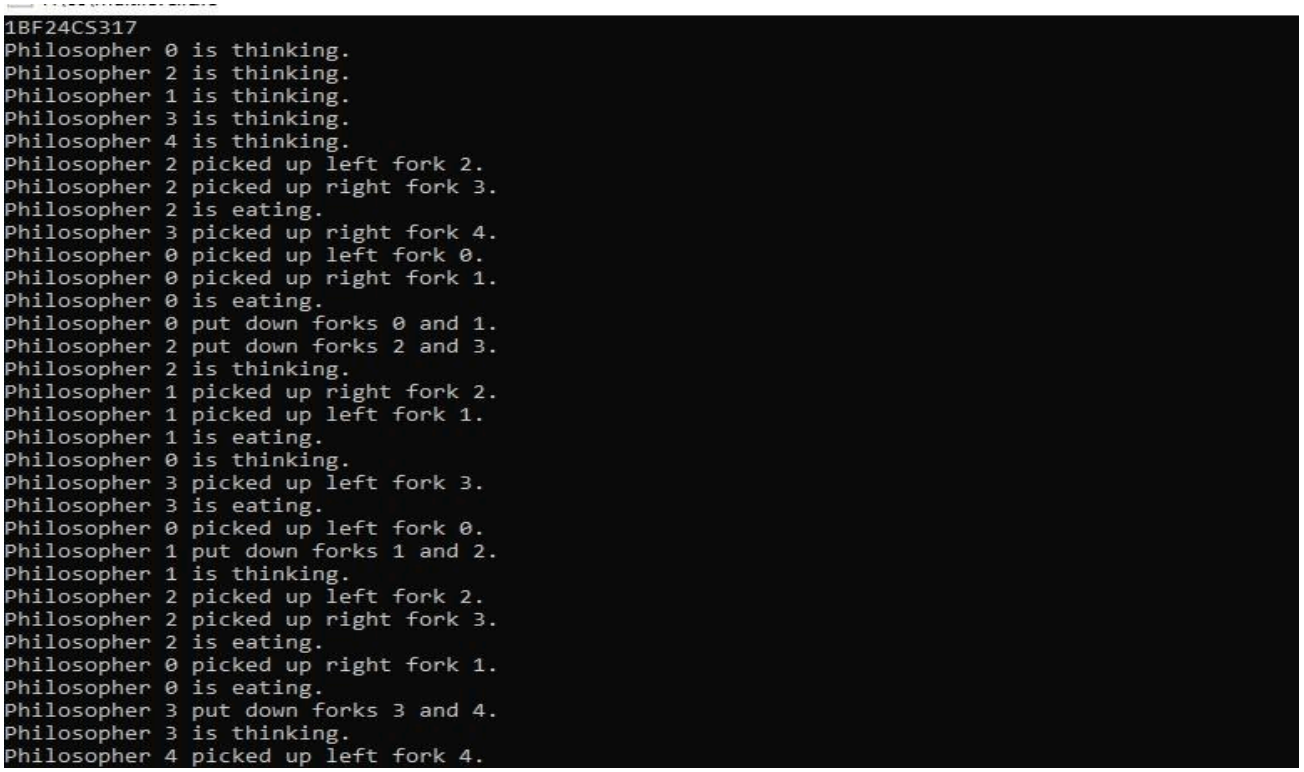
Code:

#include<stdio.h>

```
#include<pthread.h>
#include<unistd.h>
#define N 5
pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];
void* philosopher(void* num)
{
int id = *(int*)num;
int left = id;
int right = (id + 1) % N;
while (1)
{
printf("Philosopher %d is thinking.\n", id);
sleep(1);
if (id % 2 == 0)
{
pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);

pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right); }
else
{
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right); // Pick up left
fork pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);
}

printf("Philosopher %d is eating.\n", id);
sleep(2);
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}
return NULL;
}

int main() {
int i;
int ids[N];
printf("1BF24CS317");
```

```
for (i = 0; i < N; i++)
{ pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
}
// Create philosopher threads
for (i = 0; i < N; i++)

{
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
} // Join threads (will never happen here, but good practice)
for (i = 0; i < N; i++)
{
pthread_join(philosophers[i], NULL); }
// Destroy mutexes (unreachable here)
for (i = 0; i < N; i++)
{
pthread_mutex_destroy(&forks[i]);
}
return 0;
}
```

Result:



Program 7

Question:
Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

Code:

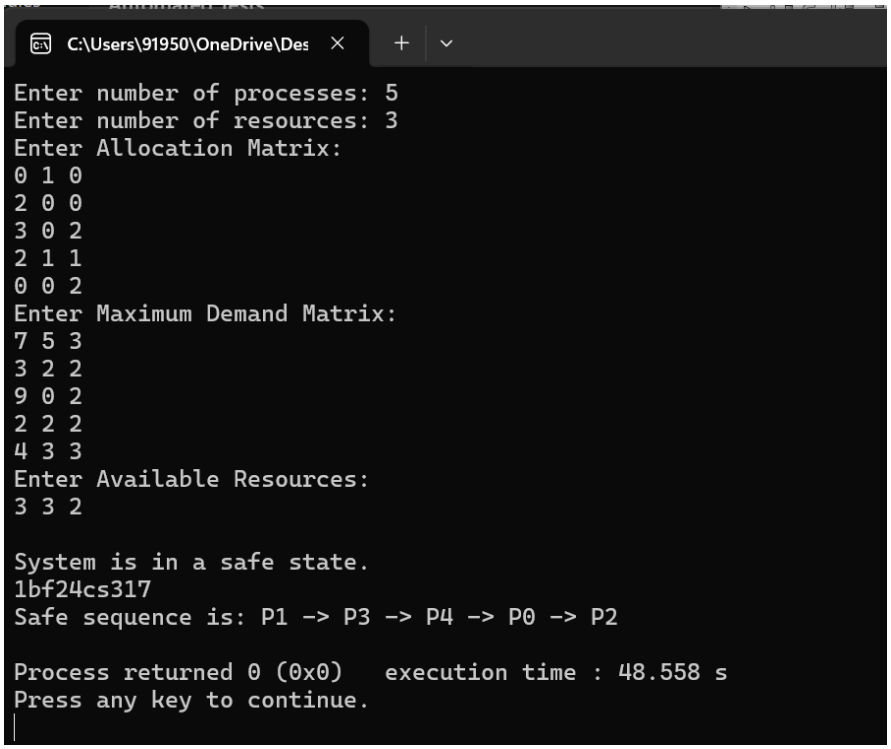
```
#include <stdio.h>

int main() {
   int n, m, i, j, k;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);
```

```
int alloc[n][m], max[n][m], need[n][m];
int avail[m];

printf("Enter Allocation Matrix:\n");
for(i = 0; i < n; i++) {
   for(j = 0; j < m; j++) {
      scanf("%d", &alloc[i][j]);
   }
}

printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++) {
   for(j = 0; j < m; j++) {
      scanf("%d", &max[i][j]);
   }
}

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++) {
   scanf("%d", &avail[i]);
}
for(i = 0; i < n; i++) {
   for(j = 0; j < m; j++) {
      need[i][j] = max[i][j] - alloc[i][j];
   }
}

int finish[n], safeSeq[n];
int work[m];
for(i = 0; i < m; i++)
   work[i] = avail[i];
for(i = 0; i < n; i++)
   finish[i] = 0;
int count = 0;
while(count < n) {
   int found = 0;
   for(i = 0; i < n; i++) {
      if(finish[i] == 0) {
         int possible = 1;
         for(j = 0; j < m; j++) {
            if(need[i][j] > work[j]) {
               possible = 0;
```

```
                break;
            }
        }
        if(possible) {

            for(k = 0; k < m; k++) {
                work[k] += alloc[i][k];
            }

            safeSeq[count] = i;
            count++;

            finish[i] = 1;
            found = 1;
        }
    }
  }

  if(found == 0) {
    break;
  }
}

if(count == n) {
  printf("\nSystem is in a safe state.\n");
  printf("1bf24cs317\n");
  printf("Safe sequence is: ");

  for(i = 0; i < n; i++) {
    printf("P%d", safeSeq[i]);

    if(i != n - 1)
      printf(" -> ");
  }

  printf("\n");
}
else {
  printf("\nSystem is NOT in safe state.\n");
}

return 0;
```

}

Result:


C:\Users\91950\OneDrive\Des
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2
System is in a safe state.
1bf24cs317
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0) execution time : 48.558 s
Press any key to continue.